Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

З ЛАБОРАТОРНОЇ РОБОТИ № 2

з дисципліни «Професійний Python»
на тему: «Структури даних та функціональна декомпозиція»

Виконала:
студентка 3 курсу
групи ФеП-33
Наборська Дарія
Русланівна

Перевірив:
професор Свелеба Сергій
Андрійович

Львів-2026

Тема роботи

Структури даних та функціональна декомпозиція.

Мета роботи

Сформувати практичні навички професійного використання базових та розширених структур даних Python, навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій та декомпозувати програму на невеликі функції з чітко визначеною відповідальністю.

Номер індивідуального варіанта

Варіант 9. Аналіз обліку спортивних результатів.

Постановка задачі

Створити модуль обробки структурованих даних про спортсменів. Для кожного спортсмена зберігати: ID, ім'я, вид спорту, результат (у балах) та категорію. Реалізувати:

- унікальні види спорту;
- групування спортсменів за видом спорту;
- найкращий результат;
- середній результат;
- формування рейтингу;
- фільтрацію за категорією або результатом;
- Counter спортсменів за видами спорту;
- dict-index за спортсменом або ID.

Програма повинна використовувати list, tuple, set, dict, структури з collections, list/set/dict comprehensions, щонайменше 6 функцій, функції з *args та **kwargs, lambda, closure, decorator, а також містити аналіз складності та benchmark.

Завдання підвищеної складності

Методичка вимагає для максимальної оцінки щонайменше три додаткові можливості. Реалізовано такі:

- **Універсальна функція фільтрації** — filter_items(athletes, predicate) приймає будь-який predicate.
- **Універсальне сортування** — sort_items(athletes, key, reverse) з довільною key-функцією.
- **Параметризований decorator** — log_operation(history, label) записує назву операції в історію.

- **Closure для створення filters** — make_result_filter(minimum, maximum) та make_category_filter(*categories).
- **Вкладене групування** — group_by_sport_and_category: вид спорту → категорія → імена.
- **Статистика через Counter** — кількість спортсменів за видами спорту та найчастіша категорія (most_common).
- **deque для історії останніх operations** — deque(maxlen=5) зберігає 5 останніх операцій.
- **Benchmark list vs set/dict** — модуль benchmark.py для 1 000, 10 000 і 100 000 записів.
- **Export summary** — build_summary(athletes, **extra) формує підсумковий словник.
- **Pipeline processing** — pipeline(athletes, *steps) послідовно застосовує кроки обробки.

Репозиторій: https://github.com/dashanaborska/python_lab, каталог Lab_02 (звіт — у Lab_02/report/).

Опис структури даних

Кожен спортсмен представлений незмінним об'єктом Athlete (@dataclass(frozen=True)). Набір даних зберігається у списку list[Athlete] у модулі data.py (10 записів).

```python
from dataclasses import dataclass


@dataclass(frozen=True)
class Athlete:
    id: int
    name: str
    sport: str
    result: float
    category: str
```

Поле	Тип	Опис
id	int	унікальний ідентифікатор
name	str	ім'я та прізвище спортсмена
sport	str	вид спорту
result	float	результат у балах (0–100)
category	str	спортивна категорія (МСМК, МС, КМС, розряди)

Пояснення вибору list, tuple, set, dict

- **list** — основний набір спортсменів: порядок важливий, дані можна доповнювати, потрібен послідовний обхід, сортування та фільтрація (list[Athlete], результати filter_items, рейтинг).
- **tuple** — незмінні групи значень: перелік допустимих категорій CATEGORIES, розміри benchmark BENCHMARK_SIZES, статистика (min, max, average), пари (місце, спортсмен) у рейтингу, час benchmark Timings.
- **set** — унікальні види спорту (get_unique_sports) та множина дозволених категорій у closure make_category_filter, де перевірка in має середню складність O(1).
- **dict** — індекси за ID та за ім'ям (create_id_index, create_name_index) для пошуку за O(1), групування (sport → список), вкладене групування та export summary.

Приклад використання collections

Використано три структури з модуля collections:

- **defaultdict** — групування без перевірки наявності ключа;
- **Counter** — підрахунок спортсменів за видами спорту та категоріями, метод most_common();
- **deque** — історія останніх операцій з обмеженням maxlen=5: старі записи автоматично видаляються з початку за O(1).

```Python
@log_operation(operations, "group by sport")
def group_by_sport(athletes: list[Athlete]) -> dict[str, list[Athlete]]:
    groups: defaultdict[str, list[Athlete]] = defaultdict(list)
    for athlete in athletes:
        groups[athlete.sport].append(athlete)
    return dict(groups)


@log_operation(operations, "count by sport")
def count_by_sport(athletes: list[Athlete]) -> Counter[str]:
    return Counter(athlete.sport for athlete in athletes)
```

```Python
from collections import deque

from sport_results.config import HISTORY_SIZE

operations: deque[str] = deque(maxlen=HISTORY_SIZE)


def last_operations() -> list[str]:
    return list(operations)


def clear_history() -> None:
```

```
    operations.clear()
```

Структура проєкту

```
None
python_lab/
├── README.md
├── Lab_01/
└── Lab_02/
    ├── .gitignore
    ├── pyproject.toml
    ├── README.md
    ├── report/
    │   └── Lab_02_Naborska.pdf
    ├── src/
    │   └── sport_results/
    │       ├── __init__.py
    │       ├── analytics.py
    │       ├── benchmark.py
    │       ├── config.py
    │       ├── data.py
    │       ├── decorators.py
    │       ├── history.py
    │       ├── main.py
    │       ├── models.py
    │       └── processors.py
    └── tests/
        └── test_processing.py
```

- **models.py** — модель Athlete;
- **data.py** — вхідний набір структурованих даних;
- **config.py** — конфігураційні константи (tuple категорій, розмір історії, параметри benchmark);
- **processors.py** — пошук, фільтрація, групування, підрахунок, індекси;
- **analytics.py** — агрегація, статистика, сортування, рейтинг, *args/**kwargs, pipeline;
- **decorators.py** — decorators measure_time і log_operation;
- **history.py** — історія операцій на основі deque;
- **benchmark.py** — експериментальне порівняння пошуку;
- **main.py** — точка входу, демонстрація всіх можливостей;
- **tests/test_processing.py** — 17 тестів pytest.

Основні functions

Функція	Модуль	Призначення
get_unique_sports	processors	унікальні види спорту (set)
group_by_sport	processors	групування за видом спорту
group_by_sport_and_category	processors	вкладене групування

count_by_sport / count_by_category	processors	Counter за видами / категоріями
create_id_index / create_name_index	processors	dict-index за ID / ім'ям
find_by_name / find_linear	processors	пошук у dict / лінійний пошук у list
filter_items	processors	універсальна фільтрація
make_result_filter / make_category_filter	processors	closures для фільтрів
find_best_result	analytics	найкращий результат
calculate_average_result	analytics	середній результат
calculate_average_values	analytics	середнє з *args
result_statistics	analytics	tuple (min, max, average)
sort_items / build_rating	analytics	універсальне сортування / рейтинг
create_athlete / build_summary	analytics	**kwargs: створення запису / summary
pipeline	analytics	послідовна обробка
measure_time / log_operation	decorators	вимірювання часу / журнал операцій

src/sport_results/processors.py

```python
from collections import Counter, defaultdict
from collections.abc import Callable

from sport_results.decorators import log_operation
from sport_results.history import operations
from sport_results.models import Athlete

Predicate = Callable[[Athlete], bool]


@log_operation(operations, "unique sports")
def get_unique_sports(athletes: list[Athlete]) -> set[str]:
    return {athlete.sport for athlete in athletes}


@log_operation(operations, "group by sport")
def group_by_sport(athletes: list[Athlete]) -> dict[str, list[Athlete]]:
    groups: defaultdict[str, list[Athlete]] = defaultdict(list)
    for athlete in athletes:
        groups[athlete.sport].append(athlete)
    return dict(groups)


def group_by_sport_and_category(
```

```
    athletes: list[Athlete],
) -> dict[str, dict[str, list[str]]]:
    nested: defaultdict[str, defaultdict[str, list[str]]] = defaultdict(
        lambda: defaultdict(list)
    )
    for athlete in athletes:
        nested[athlete.sport][athlete.category].append(athlete.name)
    return {sport: dict(categories) for sport, categories in nested.items()}


@log_operation(operations, "count by sport")
def count_by_sport(athletes: list[Athlete]) -> Counter[str]:
    return Counter(athlete.sport for athlete in athletes)


def count_by_category(athletes: list[Athlete]) -> Counter[str]:
    return Counter(athlete.category for athlete in athletes)


def create_id_index(athletes: list[Athlete]) -> dict[int, Athlete]:
    return {athlete.id: athlete for athlete in athletes}


def create_name_index(athletes: list[Athlete]) -> dict[str, Athlete]:
    return {athlete.name.lower(): athlete for athlete in athletes}


@log_operation(operations, "search by name")
def find_by_name(index: dict[str, Athlete], name: str) -> Athlete | None:
    return index.get(name.strip().lower())


def find_linear(athletes: list[Athlete], athlete_id: int) -> Athlete | None:
    for athlete in athletes:
        if athlete.id == athlete_id:
            return athlete
    return None


@log_operation(operations, "filter")
def filter_items(
    athletes: list[Athlete],
    predicate: Predicate,
) -> list[Athlete]:
    return [athlete for athlete in athletes if predicate(athlete)]


def make_result_filter(minimum: float, maximum: float = 100.0) -> Predicate:
    def predicate(athlete: Athlete) -> bool:
        return minimum <= athlete.result <= maximum

    return predicate


def make_category_filter(*categories: str) -> Predicate:
    allowed = {category.lower() for category in categories}

    def predicate(athlete: Athlete) -> bool:
        return athlete.category.lower() in allowed

    return predicate
```

src/sport_results/analytics.py

```python
from collections.abc import Callable
from typing import Any

from sport_results.decorators import log_operation, measure_time
from sport_results.history import operations
from sport_results.models import Athlete


@log_operation(operations, "best result")
def find_best_result(athletes: list[Athlete]) -> Athlete | None:
    if not athletes:
        return None
    return max(athletes, key=lambda athlete: athlete.result)


@measure_time
@log_operation(operations, "average result")
def calculate_average_result(athletes: list[Athlete]) -> float:
    if not athletes:
        return 0.0
    return sum(athlete.result for athlete in athletes) / len(athletes)


def calculate_average_values(*values: float) -> float:
    if not values:
        return 0.0
    return sum(values) / len(values)


def result_statistics(athletes: list[Athlete]) -> tuple[float, float, float]:
    results = [athlete.result for athlete in athletes]
    if not results:
        return 0.0, 0.0, 0.0
    return min(results), max(results), calculate_average_values(*results)


def sort_items(
    athletes: list[Athlete],
    key: Callable[[Athlete], Any],
    reverse: bool = False,
) -> list[Athlete]:
    return sorted(athletes, key=key, reverse=reverse)


@log_operation(operations, "rating")
def build_rating(athletes: list[Athlete]) -> list[tuple[int, Athlete]]:
    ordered = sort_items(
        athletes,
        key=lambda athlete: (-athlete.result, athlete.name),
    )
    return list(enumerate(ordered, start=1))


def create_athlete(**fields: Any) -> Athlete:
    return Athlete(**fields)


def build_summary(athletes: list[Athlete], **extra: Any) -> dict[str, Any]:
    minimum, maximum, average = result_statistics(athletes)
```

```
    summary: dict[str, Any] = {
        "athletes": len(athletes),
        "sports": len({athlete.sport for athlete in athletes}),
        "min": minimum,
        "max": maximum,
        "average": round(average, 2),
    }
    summary.update(extra)
    return summary


def pipeline(
    athletes: list[Athlete],
    *steps: Callable[[list[Athlete]], list[Athlete]],
) -> list[Athlete]:
    result = athletes
    for step in steps:
        result = step(result)
    return result
```

Приклад comprehensions

```Python
unique = {athlete.sport for athlete in athletes}

index = {athlete.id: athlete for athlete in athletes}

selected = [athlete for athlete in athletes if predicate(athlete)]

average = sum(athlete.result for athlete in athletes) / len(athletes)
```

- **Set comprehension** — get_unique_sports, множина allowed у make_category_filter.
- **Dict comprehension** — create_id_index, create_name_index, перетворення вкладеного defaultdict у dict.
- **List comprehension** — filter_items, генерація даних у benchmark.
- **Generator expression** — calculate_average_result та Counter(...): проміжний список не створюється.

Приклад *args

```Python
def calculate_average_values(*values: float) -> float:
    if not values:
        return 0.0
    return sum(values) / len(values)


calculate_average_values(75, 80, 92, 96)
```

Функція приймає довільну кількість позиційних аргументів (у функції вони доступні як tuple values). *args також використано у

make_category_filter(*categories) та pipeline(athletes, *steps), а у result_statistics список результатів розпаковується: calculate_average_values(*results).

Приклад **kwargs

```python
def create_athlete(**fields: Any) -> Athlete:
    return Athlete(**fields)


create_athlete(id=11, name="Катерина Гнатюк", sport="Бокс",
               result=86.4, category="КМС")
build_summary(athletes, variant=9, author="Наборська Д. Р.")
```

**kwargs збирає іменовані аргументи у dict. У create_athlete він передається в конструктор Athlete, а в build_summary додаткові поля (variant, author) додаються до підсумкового словника.

Приклад lambda

```python
max(athletes, key=lambda athlete: athlete.result)

sort_items(athletes, key=lambda athlete: (-athlete.result, athlete.name))

pipeline(
    athletes,
    lambda items: filter_items(items, lambda a: a.sport == "Плавання"),
    lambda items: sort_items(items, key=lambda a: a.name),
)
```

Lambda використано для коротких key-функцій сортування та пошуку максимуму і для кроків pipeline. Складна логіка винесена в окремі іменовані функції.

Пояснення closure

```python
def make_result_filter(minimum: float, maximum: float = 100.0) -> Predicate:
    def predicate(athlete: Athlete) -> bool:
        return minimum <= athlete.result <= maximum

    return predicate


def make_category_filter(*categories: str) -> Predicate:
    allowed = {category.lower() for category in categories}

    def predicate(athlete: Athlete) -> bool:
        return athlete.category.lower() in allowed

    return predicate
```

Внутрішня функція predicate після завершення make_result_filter() продовжує пам'ятати змінні minimum і maximum зовнішньої функції, а predicate з make_category_filter() — множину allowed. Тому можна створювати багато готових фільтрів, наприклад make_result_filter(85), і передавати їх в універсальну filter_items.

Пояснення decorator

```python
from collections import deque
from collections.abc import Callable
from functools import wraps
from time import perf_counter
from typing import ParamSpec, TypeVar

P = ParamSpec("P")
R = TypeVar("R")


def measure_time(func: Callable[P, R]) -> Callable[P, R]:
    @wraps(func)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        start = perf_counter()
        result = func(*args, **kwargs)
        elapsed = perf_counter() - start
        print(f"[time] {func.__name__}: {elapsed:.8f} s")
        return result

    return wrapper


def log_operation(
    history: deque[str],
    label: str | None = None,
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        @wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            history.append(label or func.__name__)
            return func(*args, **kwargs)

        return wrapper

    return decorator
```

- **measure_time** — вимірює і друкує час виконання функції (застосовано до calculate_average_result).
- **log_operation(history, label)** — параметризований decorator: зовнішня функція отримує параметри, decorator — функцію, wrapper додає назву операції в deque і викликає оригінальну функцію. Він одночасно демонструє closure, *args та **kwargs.
- **functools.wraps** зберігає ім'я та атрибути оригінальної функції (перевіряється в тестах).

- **ParamSpec / TypeVar** зберігають сигнатуру декорованої функції для type checker (mypy без помилок).

Результат роботи програми

Встановлення пакета у virtual environment

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_02> python --version
Python 3.13.16
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_02> python -m pip install -e .
Installing collected packages: sport-results
  Attempting uninstall: sport-results
    Found existing installation: sport-results 0.2.0
    Uninstalling sport-results-0.2.0:
      Successfully uninstalled sport-results-0.2.0
Successfully installed sport-results-0.2.0
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_02> python -m pip list
Package           Version Editable project location
----------------- ------- --------------------------
pip               26.2.1
pytest            9.1.1
sport-results     0.2.0   C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_02
```

Рисунок 1 — Перевірка Python та встановлення пакета

Запуск програми

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_02> python -m sport_results.main

=== Усі спортсмени ===
  1  Олена Ковальчук   Плавання          87.50  КМС
  2  Максим Бондар     Легка атлетика    92.30  МС
  3  Ірина Мельник     Плавання          78.40  I розряд
  4  Андрій Шевченко   Бокс              81.00  КМС
  5  Софія Ткачук      Легка атлетика    95.10  МСМК
  6  Тарас Новак       Бокс              88.60  МС
  7  Марія Бойко       Плавання          90.20  МС
  8  Дмитро Савчук     Велоспорт         73.80  II розряд
  9  Юлія Кравець      Велоспорт         84.90  КМС
 10  Богдан Лисенко    Легка атлетика    69.50  I розряд

=== Унікальні види спорту (set) ===
Бокс, Велоспорт, Легка атлетика, Плавання
Допустимі категорії (tuple): МСМК, МС, КМС, I розряд, II розряд, III розряд

=== Групування за видом спорту (defaultdict) ===
Плавання        -> Олена Ковальчук, Ірина Мельник, Марія Бойко
Легка атлетика  -> Максим Бондар, Софія Ткачук, Богдан Лисенко
Бокс            -> Андрій Шевченко, Тарас Новак
Велоспорт       -> Дмитро Савчук, Юлія Кравець

=== Кількість спортсменів за видами спорту (Counter) ===
Плавання        3
Легка атлетика  3
Бокс            2
Велоспорт       2

=== Вкладене групування: вид спорту -> категорія ===
Плавання
    КМС       Олена Ковальчук
    I розряд  Ірина Мельник
    МС        Марія Бойко
Легка атлетика
    МС        Максим Бондар
    МСМК      Софія Ткачук
    I розряд  Богдан Лисенко
Бокс
    КМС       Андрій Шевченко
    МС        Тарас Новак
Велоспорт
    II розряд Дмитро Савчук
    КМС       Юлія Кравець
Найчастіша категорія: КМС (3)
```

Рисунок 2 — Усі спортсмени, set, tuple, групування, Counter, вкладене групування

```
=== Найкращий і середній результат ===
Найкращий результат: Софія Ткачук, Легка атлетика, 95.10
[time] calculate_average_result: 0.00000481 s
Середній результат: 84.13
Діапазон результатів (tuple): (69.5, 95.1)

=== Рейтинг спортсменів (sorted + lambda) ===
 1. Софія Ткачук        95.10
 2. Максим Бондар       92.30
 3. Марія Бойко         90.20
 4. Тарас Новак         88.60
 5. Олена Ковальчук     87.50
 6. Юлія Кравець        84.90
 7. Андрій Шевченко     81.00
 8. Ірина Мельник       78.40
 9. Дмитро Савчук       73.80
10. Богдан Лисенко      69.50

=== Фільтрація за результатом >= 85 (closure) ===
  1  Олена Ковальчук   Плавання          87.50  КМС
  2  Максим Бондар     Легка атлетика    92.30  МС
  5  Софія Ткачук      Легка атлетика    95.10  МСМК
  6  Тарас Новак       Бокс              88.60  МС
  7  Марія Бойко       Плавання          90.20  МС

=== Фільтрація за категоріями МС, МСМК (closure + *args) ===
  2  Максим Бондар     Легка атлетика    92.30  МС
  5  Софія Ткачук      Легка атлетика    95.10  МСМК
  6  Тарас Новак       Бокс              88.60  МС
  7  Марія Бойко       Плавання          90.20  МС
```

Рисунок 3 — Найкращий і середній результат (decorator measure_time), рейтинг, фільтрація

```
=== Плавання за алфавітом (pipeline) ===
  3  Ірина Мельник     Плавання          78.40  І розряд
  7  Марія Бойко       Плавання          90.20  МС
  1  Олена Ковальчук   Плавання          87.50  КМС

=== Пошук через dict-index ===
ID 7 -> Марія Бойко (Плавання, 90.2)
ID 42 -> не знайдено
'тарас новак' -> Тарас Новак (Бокс, 88.6)

=== *args та **kwargs ===
Середнє через *args: 85.75
Створено через **kwargs: Катерина Гнатюк (Бокс, 86.4)

=== Export summary (**kwargs) ===
athletes  10
sports    4
min       69.5
max       95.1
average   84.13
variant   9
author    Наборська Д. Р.

=== Останні операції (deque) ===
rating -> filter -> filter -> filter -> search by name
```

*Рисунок 4 — Pipeline, пошук через dict-index, *args, **kwargs, summary та історія deque*

Автоматизовані тести

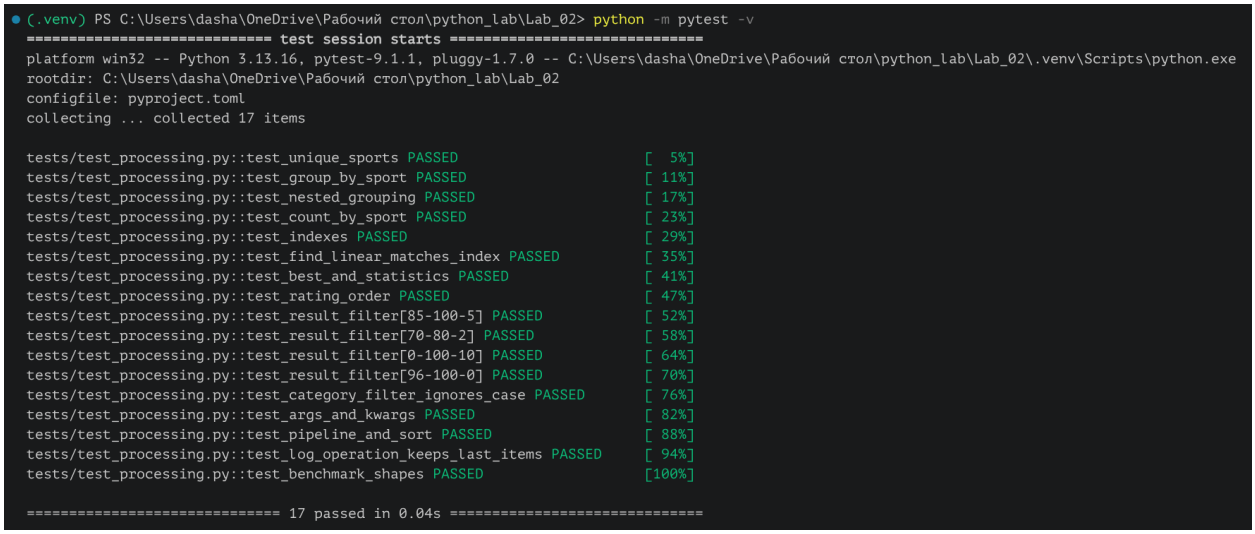


Рисунок 5 — Результат виконання тестів pytest

Таблиця складності operations

Операція в проєкті	Структура	Складність
Лінійний пошук за ID (find_linear)	list	O(n)
Побудова index (create_id_index)	dict	O(n)
Пошук за ID у index (index.get)	dict	O(1) середнє
Пошук за ім'ям (find_by_name)	dict	O(1) середнє
Перевірка категорії в allowed	set	O(1) середнє
Унікальні види спорту	set	O(n)
Групування / Counter	defaultdict / Counter	O(n)
Фільтрація (filter_items)	list	O(n)
Найкращий результат (max)	list	O(n)
Рейтинг (sorted)	list	O(n log n)
Додавання в історію (append з maxlen)	deque	O(1)

Лінійний пошук у list у гіршому випадку переглядає всі n елементів. Побудова dict-index потребує одного проходу O(n), але після цього кожен пошук за ключем виконується в середньому за O(1) завдяки хешуванню. Сортування за допомогою Timsort має складність O(n log n), тому рейтинг будується один раз, а не при кожному запиті.

Результати benchmark

Для 1 000, 10 000 і 100 000 згенерованих записів виміряно середній час одного пошуку останнього ID (найгірший випадок для list) за 200 повторень (timeit).

```
● (.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_02> sport-results-benchmark
  Час одного пошуку останнього ID, мкс
     Записів |  list search |  dict search |  x in list |  x in set |  list/dict
  ------------------------------------------------------------------------------
       1,000 |        27.92 |        0.091 |       8.37 |     0.071 |       307x
      10,000 |       322.44 |        0.090 |      94.39 |     0.119 |      3591x
     100,000 |      3260.19 |        0.092 |    1002.32 |     0.076 |     35423x
```

Рисунок 6 — Результат benchmark пошуку у list, dict та set

Кількість records	list search, мкс	dict search, мкс	x in list, мкс	x in set, мкс
1 000	27.92	0.091	8.37	0.071
10 000	322.44	0.090	94.39	0.119
100 000	3260.19	0.092	1002.32	0.076

Час пошуку в list зростає приблизно в 10 разів при збільшенні кількості записів у 10 разів, що відповідає O(n). Час пошуку в dict та перевірки в set майже не змінюється (≈0,1 мкс), що відповідає O(1). На 100 000 записах пошук через dict-index швидший за лінійний пошук у десятки тисяч разів. Отже, теоретична складність добре узгоджується з практичним часом виконання: для частих пошуків варто один раз побудувати індекс.

Python

```
def measure_search(count: int, repeats: int) -> Timings:
    athletes = generate_athletes(count)
    index = create_id_index(athletes)
    ids_list = [athlete.id for athlete in athletes]
    ids_set = set(ids_list)
    target = count
    list_time = timeit(lambda: find_linear(athletes, target), number=repeats)
    dict_time = timeit(lambda: index.get(target), number=repeats)
    in_list_time = timeit(lambda: target in ids_list, number=repeats)
    in_set_time = timeit(lambda: target in ids_set, number=repeats)
    return (
        to_microseconds(list_time, repeats),
        to_microseconds(dict_time, repeats),
        to_microseconds(in_list_time, repeats),
        to_microseconds(in_set_time, repeats),
    )
```

Результат git status

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_02> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Рисунок 7 — Результат git status

Результат git log --oneline

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab\Lab_02> git log --oneline
22cff9e (HEAD -> main, origin/main, origin/HEAD) Add README for Lab_02
e45e8a7 Add tests for data processing
e67eca5 Add application entry point
aa221f7 Add list vs dict search benchmark
c534c21 Add data processors and analytics
8e3512e Add decorators and operation history
9d9527c Add Lab_02 project structure and dataset
570736b Delete Lab_01/report/Lab_01_Naborska.docx
55cf829 Update laboratory report
2d3a92b Move laboratory work 1 into Lab_01 directory
1c79326 Add laboratory report
f03a7bb Move project to repository root
03cfd75 Add README with usage example
27f6d66 Add tests for services and validators
613fe8f Add menu-driven application entry point
f639e56 Add business logic and input validation
ec6d8cd Add Athlete model and custom exceptions
c99dca7 Create project structure
```

Рисунок 8 — Результат git log --oneline

Відповіді на контрольні запитання

1. Що таке list?

Впорядкована змінювана колекція елементів довільного типу з доступом за індексом.

2. Чим list відрізняється від tuple?

list можна змінювати (додавати, видаляти, сортувати елементи), tuple після створення змінити не можна; tuple легший, hashable (якщо елементи hashable) і підходить для фіксованих груп значень.

3. Чому tuple називається immutable?

Після створення не можна змінити, додати чи видалити його елементи — будь-яка «зміна» створює новий tuple.

4. Для чого використовується set?

Для зберігання унікальних елементів, видалення повторів, швидкої перевірки належності та операцій над множинами (|, &, -, ^).

5. Яка властивість елементів set є важливою?

Елементи мають бути hashable (незмінні з незмінним хешем), наприклад str, int, tuple.

6. Що таке dict?

Колекція пар key → value зі швидким доступом до значення за ключем; зберігає порядок вставки.

7. Що таке key-value structure?

Структура, у якій кожне значення пов'язане з унікальним ключем, за яким його знаходять.

8. Чому пошук key у dict зазвичай швидкий?

dict — це хеш-таблиця: за хешем ключа одразу обчислюється позиція, тому пошук у середньому O(1) без перебору всіх елементів.

9. Що таке hashability?

Властивість об'єкта мати незмінний хеш протягом життя (метод __hash__) і коректне порівняння __eq__; тільки такі об'єкти можуть бути ключами dict та елементами set.

10. Чому list не може бути key у dict?

list змінюваний, тому його хеш міг би змінитися після вставки, і ключ неможливо було б знайти; через це list не hashable.

11. Для чого використовується Counter?

Для підрахунку кількості появ елементів; має метод most_common().

12. Для чого використовується defaultdict?

Автоматично створює значення за замовчуванням для нового ключа, що спрощує групування та накопичення.

13. Для чого використовується deque?

Двостороння черга з додаванням і видаленням з обох кінців за O(1); з maxlen зберігає лише останні N елементів.

14. Що таке list comprehension?

Компактний вираз [expr for x in items if cond] для створення нового списку.

15. Що таке set comprehension?

Вираз {expr for x in items} для створення множини унікальних значень.

16. Що таке dict comprehension?

Вираз {key: value for x in items} для створення словника, наприклад індексу.

17. Що таке generator expression?

Вираз (expr for x in items), що лениво генерує значення по одному без створення проміжного списку.

18. Що означає O(1)?

Константна складність: час не залежить від кількості даних.

19. Що означає O(n)?

Лінійна складність: час зростає пропорційно кількості елементів.

20. Що означає O(n log n)?

Час зростає як n·log n; типовий для ефективних алгоритмів сортування.

21. Що означає $O(n^2)$?

Квадратична складність: час зростає пропорційно квадрату n, наприклад вкладені цикли по всіх парах елементів.

22. Яка приблизна складність пошуку у list?

O(n).

23. Яка середня складність пошуку у set?

O(1).

24. Яка середня складність пошуку key у dict?

O(1).

25. Чому правильна структура даних може прискорити програму?

Різні структури мають різну складність операцій; наприклад, заміна лінійного пошуку в list на dict-index зменшує час з O(n) до O(1), що на 100 000 записах дало прискорення в десятки тисяч разів.

26. Що таке функціональна декомпозиція?

Розбиття програми на невеликі функції, кожна з яких виконує одну логічну задачу.

27. Які переваги малих functions?

Їх легше читати, тестувати, повторно використовувати й змінювати; помилки локалізуються в одному місці.

28. Що таке positional argument?

Аргумент, що передається за позицією: f(1, 2).

29. Що таке keyword argument?

Аргумент, що передається за іменем параметра: f(minimum=85).

30. Для чого використовується *args?

Для прийому довільної кількості позиційних аргументів, які зберігаються у tuple.

31. Для чого використовується **kwargs?
Для прийому довільних іменованих аргументів, які зберігаються у dict.

32. Що таке lambda function?
Коротка анонімна функція з одного виразу: lambda x: x.result.

33. Де доцільно використовувати lambda?
Для коротких key-функцій у sorted, max, min, простих predicate та кроків pipeline.

34. Чому складну business logic не варто записувати в lambda?
lambda не має імені, документації й може містити лише один вираз, тому складну логіку важко читати, тестувати та налагоджувати.

35. Що таке closure?
Внутрішня функція, яка пам'ятає змінні зовнішньої функції після завершення її виконання.

36. Які variables може пам'ятати closure?
Параметри та локальні змінні зовнішньої (enclosing) функції, які використовуються у внутрішній функції; змінити їх можна через nonlocal.

37. Що таке decorator?
Функція, яка приймає іншу функцію і повертає нову з додатковою поведінкою, не змінюючи основний код.

38. Для чого застосовуються decorators?
Для логування, вимірювання часу, кешування, перевірки прав і аргументів, повторних спроб тощо.

39. Для чого використовується functools.wraps?
Копіює ім'я, docstring та інші атрибути оригінальної функції у wrapper, щоб декорована функція виглядала як оригінальна.

40. Чому потрібно оцінювати складність алгоритму?
Щоб передбачити, як програма поводитиметься при зростанні обсягу даних, і вчасно обрати ефективніший алгоритм або структуру даних.

Висновок

У ході лабораторної роботи на основі проєкту sport_results створено модуль обробки структурованих даних про спортсменів (варіант 9). Використано list для основного набору даних, tuple для незмінних груп значень, set для унікальних видів спорту та швидкої перевірки належності, dict для індексів, групування та summary, а також defaultdict, Counter і deque з модуля collections. Програму декомпозовано на невеликі функції, застосовано comprehensions,

generator expressions, *args, **kwargs, lambda, closures для фільтрів і decorators (зокрема параметризований). Аналіз складності та benchmark показали, що пошук у list зростає лінійно, а пошук через dict-index і set залишається практично сталим, що підтверджує теоретичні оцінки O(n) та O(1). Код покрито 17 тестами pytest і збережено в Git у вигляді логічних комітів.